
Author

Name : **PRAVEENA N**

Roll Number : **22f3001454**

Email : **22f3001454@ds.study.iitm.ac.in**

I'm a student and aspiring app developer passionate about building responsive and userfriendly applications.

Question Statement

Quiz Master - V2

It is a multi-user app (one requires an administrator and other users) that acts as an exam preparation site for multiple courses.

Description

Quiz-It is a multi-user quiz application with two roles: **Admin** and **User**. Admins can create and manage quizzes across various knowledge domains. Users can participate in multiple quizzes and view detailed performance summaries to track and improve their knowledge.

Technologies used

- SQLite for data storage
- Flask for API
- Flask security for Authentication
- VueJS for UI
- VueJS Advanced with CLI
- Bootstrap for HTML generation and styling
- SQLite for database
- Redis for caching
- Redis and Celery for batch jobs
- ChartJS for Visualization

DB Schema Design

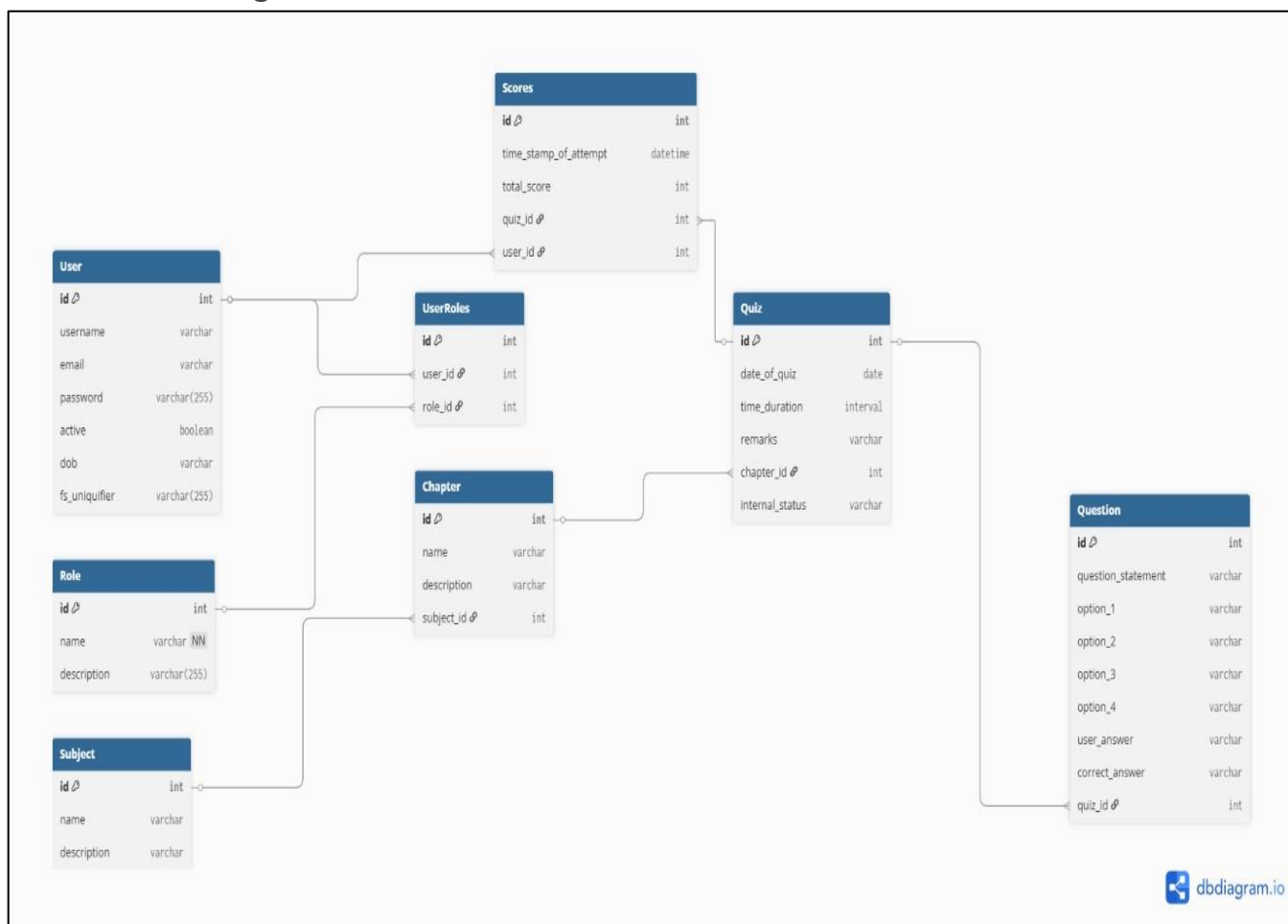


Figure -1 ER Diagram

[Click here](#) : To view the database scheme.

Reasons Behind the Database Design

1. **A hierarchical one-to-many structure was adopted** between *Subject*, *Chapter*, *Quiz*, and *Question* to reflect the natural organization of academic content. This ensured clarity, modularity, and avoided redundancy in quiz structuring.
2. **A one-to-many relationship between Quiz and Question was implemented** to allow each quiz to comprise multiple questions while maintaining question-level reusability and clean separation of content.
3. **An association table was introduced between User and Role** to facilitate a many-to-many relationship as required by SQLAlchemy. This enabled users to hold multiple roles (e.g., student, admin) and ensured role-based access control could be effectively managed.
4. **A separate Score table was incorporated** to record user quiz performance. This design choice enabled multiple users to attempt the same quiz and preserved the history of each user's attempts without overwriting previous data.
5. **A one-to-many relationship was established between Quiz and Score** to allow a quiz to be attempted by multiple users. This also facilitated user access to previously attempted quizzes, supporting performance tracking and review functionality.

API Design

Entity	Endpoint	HTTP Method	Description
Subject	/api/subject/get	GET	Get all subjects
Subject	/api/subject/create	POST	Create a new subject
Subject	/api/subject/update/<int:sub_id>	PUT	Update a subject
Subject	/api/subject/delete/<int:sub_id>	DELETE	Delete a subject
Chapter	/api/chapter/get/<int:sub_id>	GET	Get all chapters under a subject
Chapter	/api/chapter/create/<int:sub_id>	POST	Create a chapter under a subject
Chapter	/api/chapter/update/<int:chapter_id>	PUT	Update a chapter
Chapter	/api/chapter/delete/<int:chapter_id>	DELETE	Delete a chapter
Quiz	/api/quiz/get/<int:chapter_id>	GET	Get all quizzes under a chapter
Quiz	/api/quiz/create/<int:chapter_id>	POST	Create a quiz under a chapter
Quiz	/api/quiz/update/<int:quiz_id>	PUT	Update a quiz
Quiz	/api/quiz/delete/<int:quiz_id>	DELETE	Delete a quiz
Question	/api/question/get/<int:quiz_id>	GET	Get all questions under a quiz
Question	/api/question/create/<int:quiz_id>	POST	Create a question under a quiz
Question	/api/question/update/<int:question_id>	PUT	Update a question
Question	/api/question/delete/<int:question_id>	DELETE	Delete a question
Score	/api/score/get/	GET	Get all scores
Score	/api/score/create/<int:user_id>/<int:quiz_id>	POST	Submit score for a user and quiz
Score	/api/score/update/<int:score_id>	PUT	Update a score
Score	/api/score/delete/<int:score_id>	DELETE	Delete a score
User Quiz	/api/user/quiz/get/	GET	Get user quizzes
User Quiz	/api/user/scoreUpdate/create/<int:quiz_id>	POST	Update user score after quiz
User Question	/api/user/question/get/<int:quiz_id>	GET	Get questions for user quiz

Table -1 API End Points

** More end points are defined in the routes.py.**

Architecture and Features

Backend Design

The backend components were placed under the `backend/` directory. Within the application/ module:

- All controllers and route logic were defined in the `views.py` file.
- Database models and entity relationships were declared using SQLAlchemy in `models.py`.
- Configuration parameters and database connection logic were encapsulated in `config.py` and `database.py`.
- Email services were handled using the `mail.py` module.
- Background job execution was facilitated using **Celery**, initialized in `celery_init.py` and implemented via `tasks.py`.

Templates for server-side rendering were stored in the `templates/` folder (e.g., `index.html`, `mail_details.html`), while static resources such as reports were placed under `static/`. A local SQLite database was used and stored within the `instance/` directory as `quiz.db`.

Frontend Design

The frontend was developed using the **Vue.js** framework and organized under the `frontend/` directory. The `src/` folder contained:

- `components/` for modular UI elements,
- `views/` for full-page components,
- and `router/` for handling client-side navigation.

The application was initialized using `App.vue` and `main.js`, while static public assets (e.g., `favicon.ico`) were placed in the `public/` folder. Project settings and dependencies were managed through configuration files such as `vite.config.js` and `package.json`.

Features Implemented

1. Hierarchical Content Structure

A normalized one-to-many relationship was designed between **Subject**, **Chapter**, **Quiz**, and **Question**. This ensured scalable storage of educational content and logical navigation.

2. User Roles and Permissions

A many-to-many relationship between users and roles was established using an association table, as required by SQLAlchemy. This allowed flexible assignment of roles like user and admin.

3. Quiz Attempt Tracking

A trio relationship between **User**, **Score**, and **Quiz** was used to track quiz attempts. Multiple scores per user were preserved, and quiz history could be accessed for review.

4. Email Notification System

Automated email notifications were implemented using Flask-Mail, with dynamic content rendered from `mail_details.html`.

5. Asynchronous Task Processing

Background jobs such as sending emails and generating reports were executed using Celery, ensuring a non-blocking user experience.

6. Modern Client Interface

The frontend interface was developed as a responsive **Single Page Application (SPA)** using Vue.js, providing smooth navigation and an interactive user experience through component-based design.

Video

Link : https://drive.google.com/file/d/1nXM1wYD1NbT_lxFr0uEK0AvRPZ84mEOp/view?usp=drive_link